

Exercise 3-1: Create some plots

Did you use a language model?

No, I did not use an LLM for code as I followed along the ppt and experimented BUT I used an LLM for writing comments.

What did you do to verify that your code worked as expected, whether you used a language model or not?

Referred to the PPT for code.

```
In [3]: # Import the pandas library and alias it as 'pd' for convenient access to its fu
import pandas as pd
```

Get the data

```
In [4]: # Load the preprocessed mortality data from a pickle file using pandas
mortality_data = pd.read_pickle('mortality_prepped.pkl')
# Display the first five rows of the loaded mortality data to get a quick overvi
mortality_data.head()
```

```
Out[4]:
```

	Year	AgeGroup	DeathRate	MeanCentered
0	1900	01-04 Years	1983.8	1790.87584
1	1901	01-04 Years	1695.0	1502.07584
2	1902	01-04 Years	1655.7	1462.77584
3	1903	01-04 Years	1542.1	1349.17584
4	1904	01-04 Years	1591.5	1398.57584

```
In [5]: mortality_wide = pd.read_pickle('mortality_wide.pkl')
# Display the first five rows of the loaded mortality_wide data to get a quick o
mortality_wide.head()
# Display the first five rows of the loaded mortality data to get a quick overvi
```

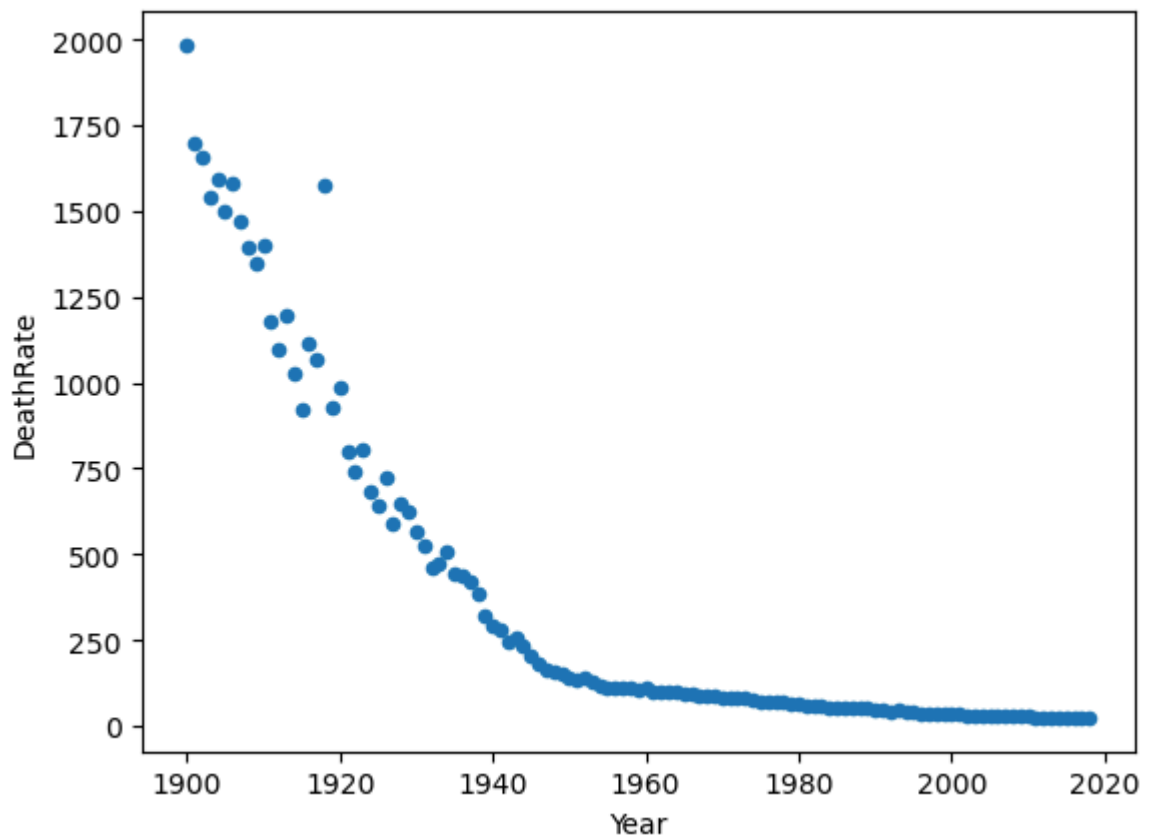
Out[5]: **AgeGroup** 01-04 Years 05-09 Years 10-14 Years 15-19 Years

Year				
1900	1983.8	466.1	298.3	484.8
1901	1695.0	427.6	273.6	454.4
1902	1655.7	403.3	252.5	421.5
1903	1542.1	414.7	268.2	434.1
1904	1591.5	425.0	305.2	471.4

Visualize the data

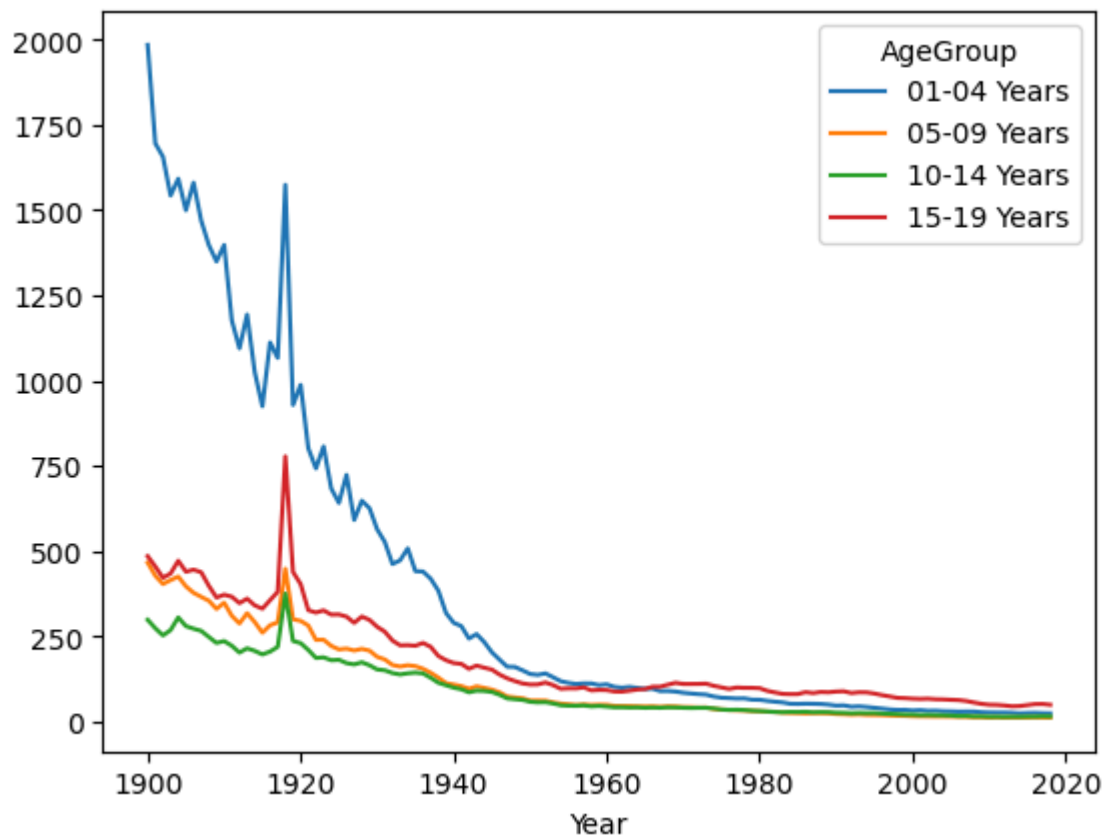
```
In [6]: # Filter the mortality_data for entries where the AgeGroup is "01-04 Years", the
# with 'Year' on the x-axis and 'DeathRate' on the y-axis to analyze trends over
mortality_data.query('AgeGroup == "01-04 Years"').plot.scatter(x='Year', y='Deat
```

Out[6]: <Axes: xlabel='Year', ylabel='DeathRate'>



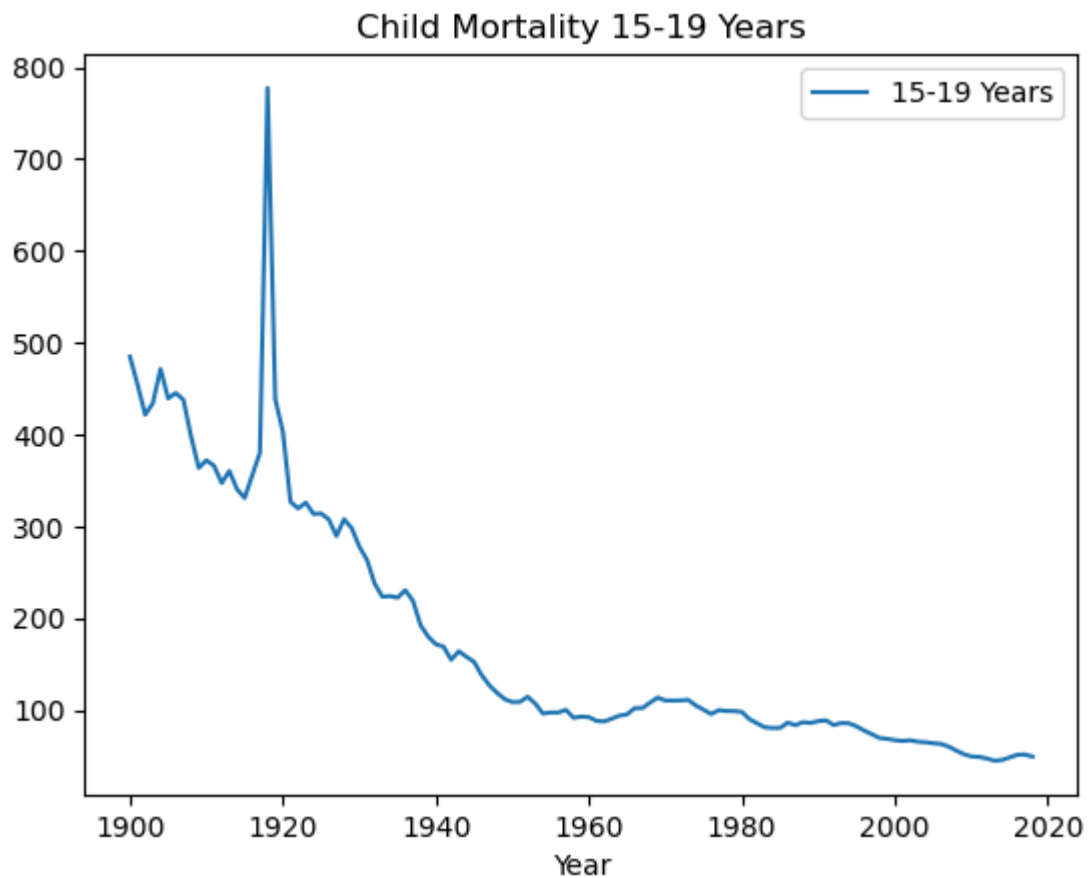
```
In [14]: # Plot all columns of the 'mortality_wide' DataFrame against the index using a l
# generate a line plot for each column in the DataFrame, providing a visual comp
# different columns over the index values (typically time or another ordered seq
mortality_wide.plot()
```

Out[14]: <Axes: xlabel='Year'>



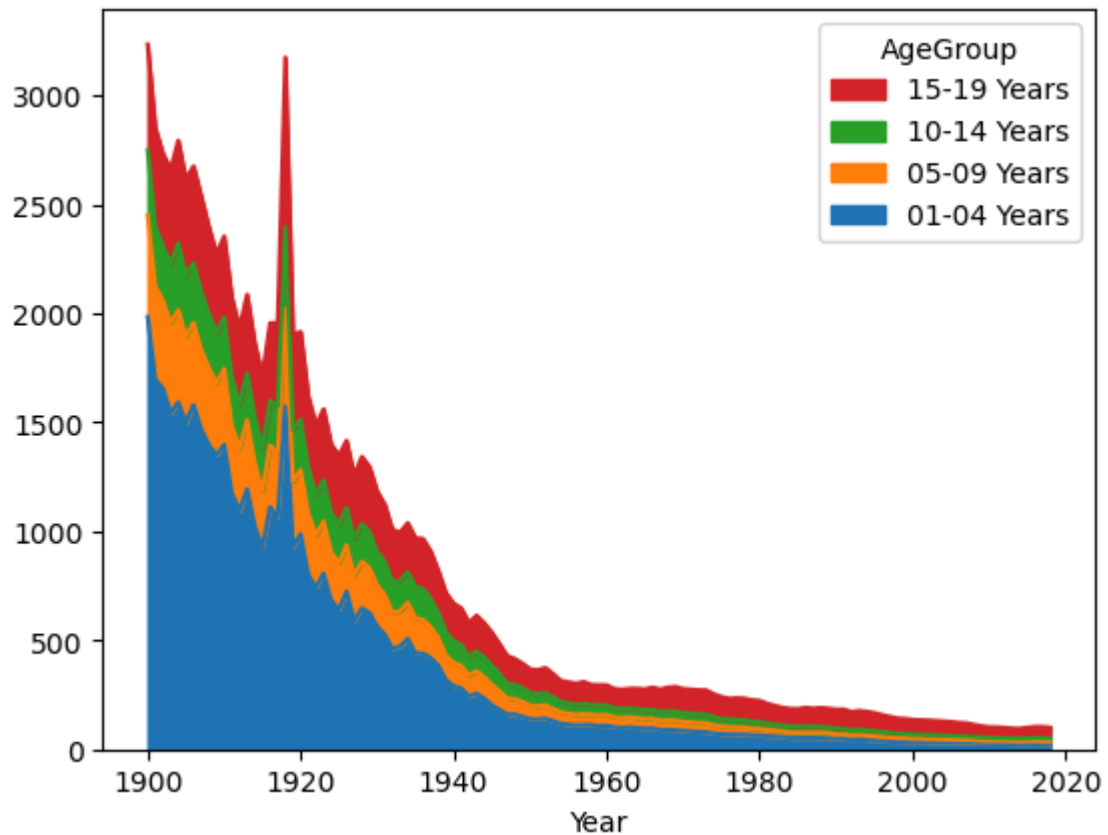
```
In [15]: # Plot a line graph for the '15-19 Years' column from the 'mortality_wide' DataF
# 'Child Mortality 15-19 Years' to specifically highlight mortality trends withi
# is disabled (Legend='False') for a cleaner visual presentation, focusing solel
mortality_wide.plot.line(y='15-19 Years', title= 'Child Mortality 15-19 Years',
```

```
Out[15]: <Axes: title={'center': 'Child Mortality 15-19 Years'}, xlabel='Year'>
```



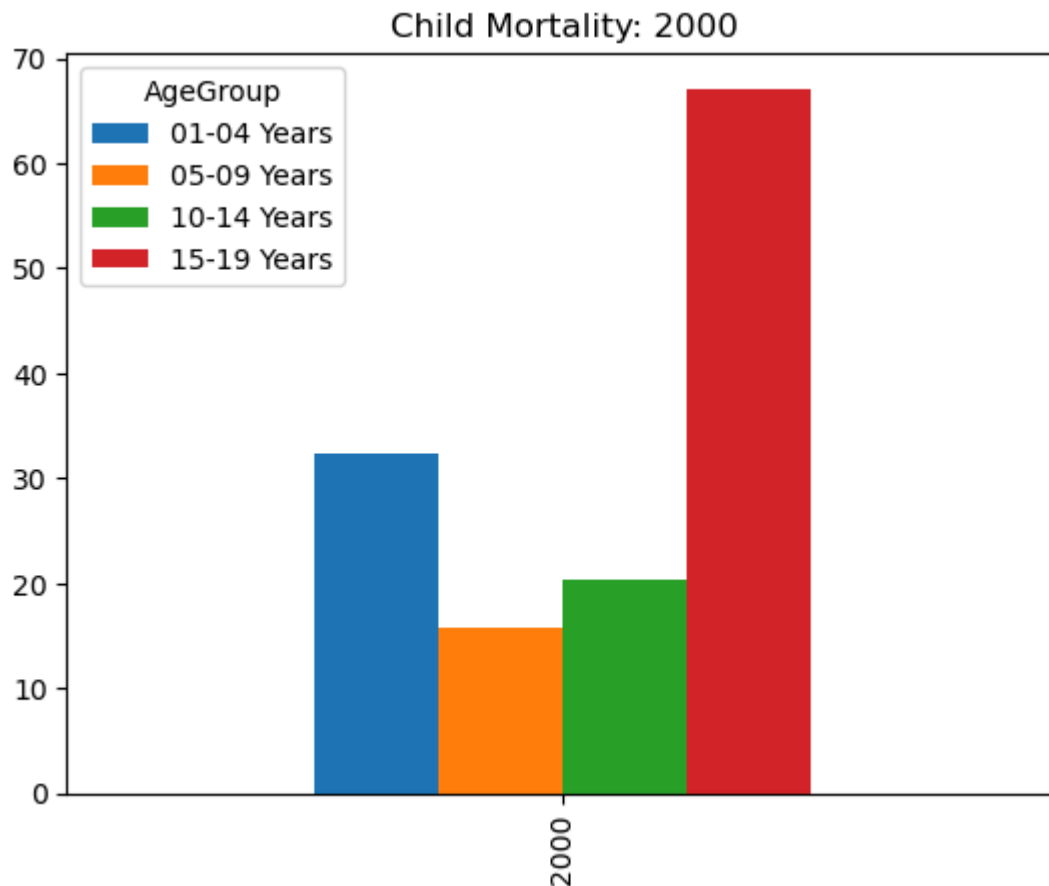
```
In [16]: # Generate an area plot for the 'mortality_wide' DataFrame, which will stack the
# column on top of one another. The 'legend = 'reverse'' option is used to rever
# ensuring the legend matches the stacking order in the plot for easier interpre
mortality_wide.plot.area(legend = 'reverse')
```

Out[16]: <Axes: xlabel='Year'>



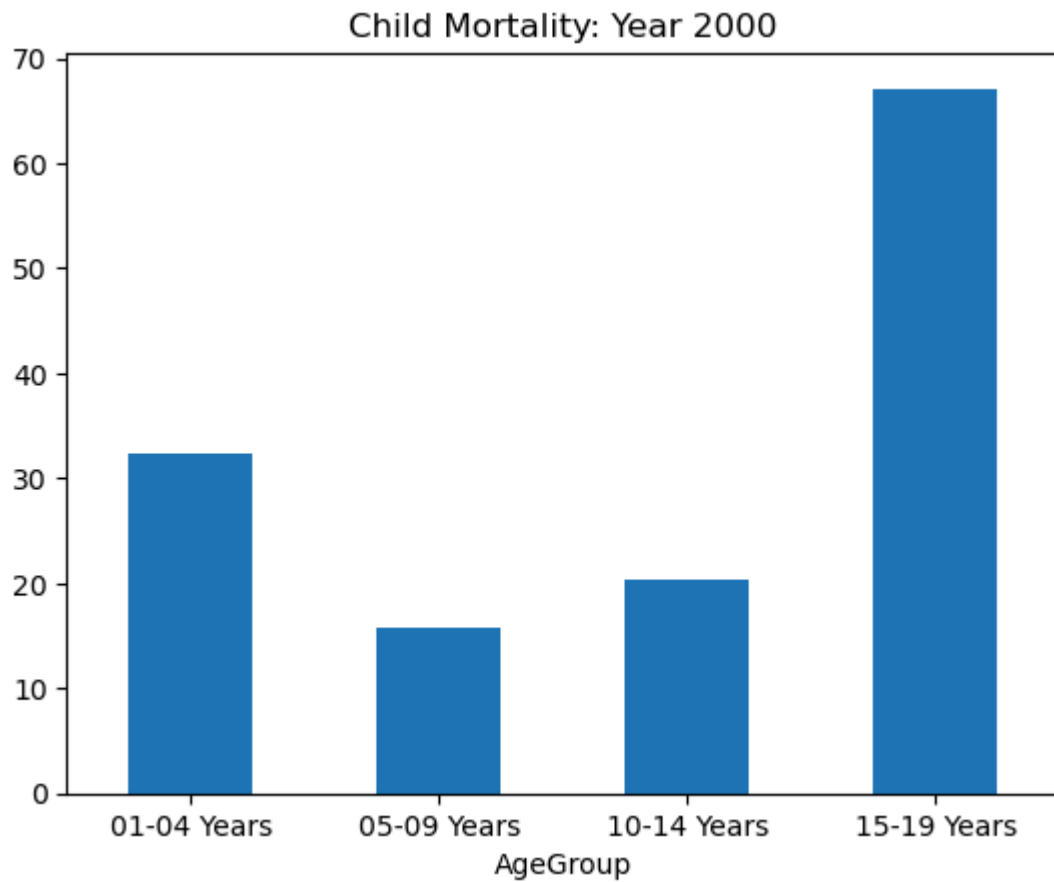
```
In [19]: # Filter 'mortality_wide' for the year 2000, then plot a bar chart with the titl
# The x-axis label is removed for clarity.
mortality_wide.query('Year == 2000')\
.plot.bar(title='Child Mortality: 2000', xlabel = '')
```

Out[19]: <Axes: title={'center': 'Child Mortality: 2000'}>



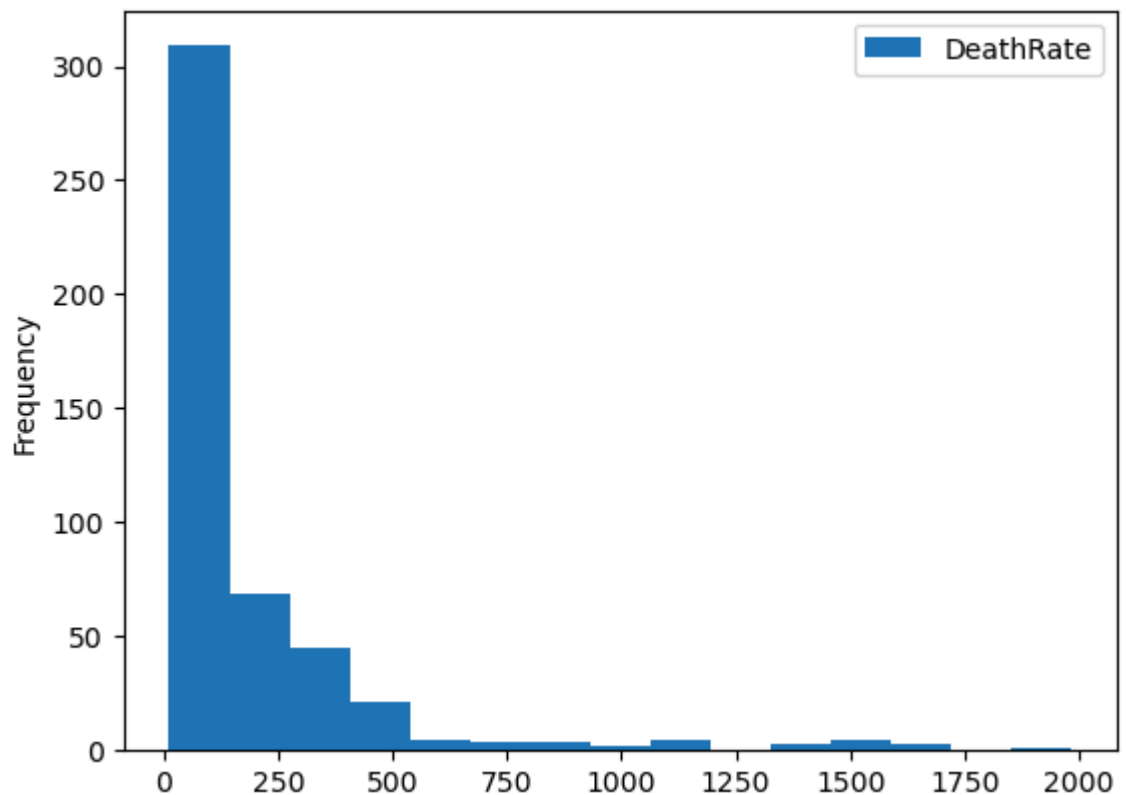
```
In [27]: # Filter the dataset for the year 2000, pivot to reorganize data with AgeGroup as
# and DeathRate as values. Then, plot a bar chart titled 'Child Mortality: Year
# with horizontal x-axis labels (rot=0) for clear age group identification.
mortality_data.query('Year == 2000') \
    .pivot(index='AgeGroup', columns='Year', values='DeathRate') \
    .plot.bar(title='Child Mortality: Year 2000', legend=False, rot=0)
```

```
Out[27]: <Axes: title={'center': 'Child Mortality: Year 2000'}, xlabel='AgeGroup'>
```



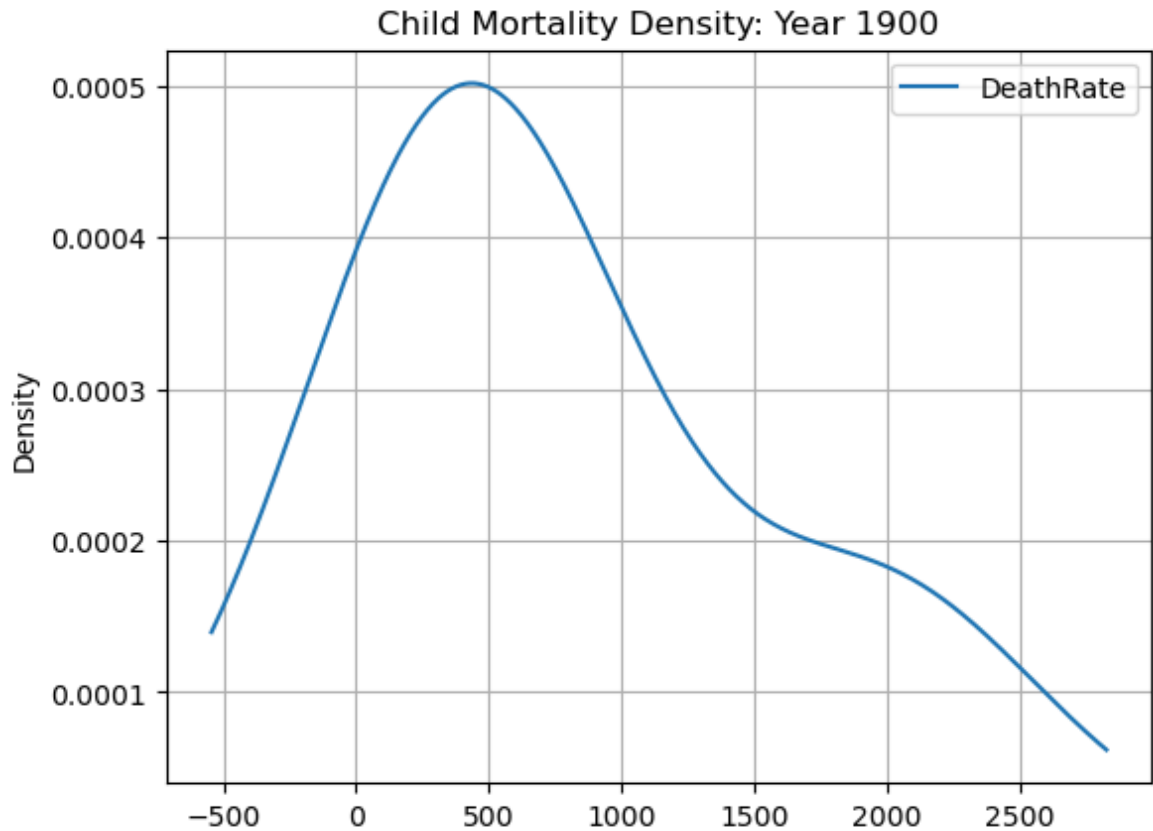
```
In [44]: # Plot a histogram of the 'DeathRate' column from 'mortality_data' using 15 bins
# distribution of death rates across the dataset, helping identify common ranges
mortality_data.plot.hist(y='DeathRate', bins=15)
```

```
Out[44]: <Axes: ylabel='Frequency'>
```



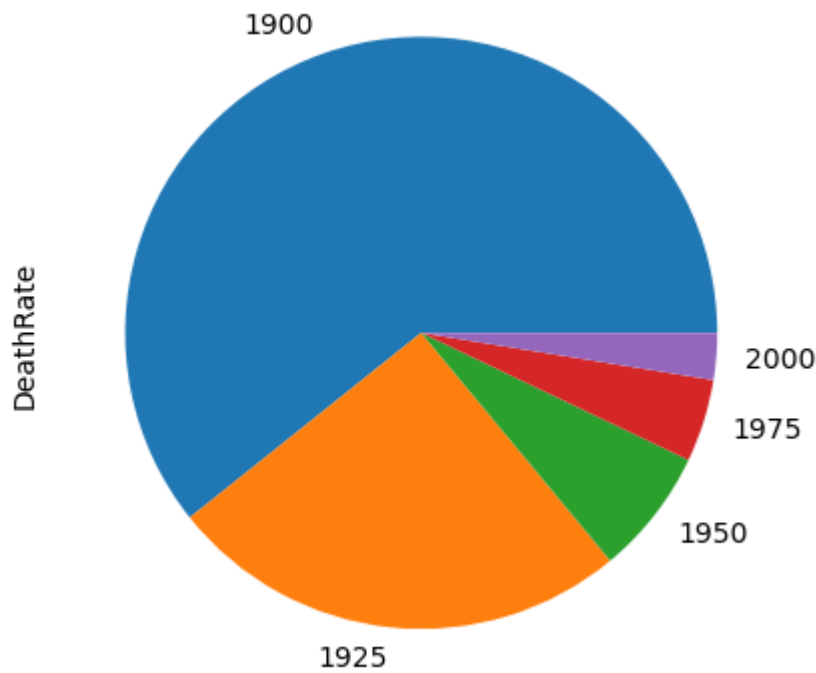
```
In [31]: # For the year 1900, this line plots the density (a smoothed, continuous version
# from 'mortality_data'. It includes a grid for easier reading and is titled 'Ch
# helping visualize the distribution and concentration of death rates for that y
mortality_data.query('Year==1900') \
    .plot.density(y='DeathRate', grid=True, title='Child Mortality Density: Year
```

```
Out[31]: <Axes: title={'center': 'Child Mortality Density: Year 1900'}, ylabel='Densit
y'>
```



```
In [32]: # This code filters 'mortality_data' for the years 1900, 1925, 1950, 1975, and 2
# It sums up the 'DeathRate' for each year and plots these sums in a pie chart.
# death rates across these selected years, illustrating changes or trends in chi
mortality_data.query('Year in (1900, 1925, 1950, 1975, 2000)') \
    .groupby('Year').DeathRate.sum().plot.pie()
```

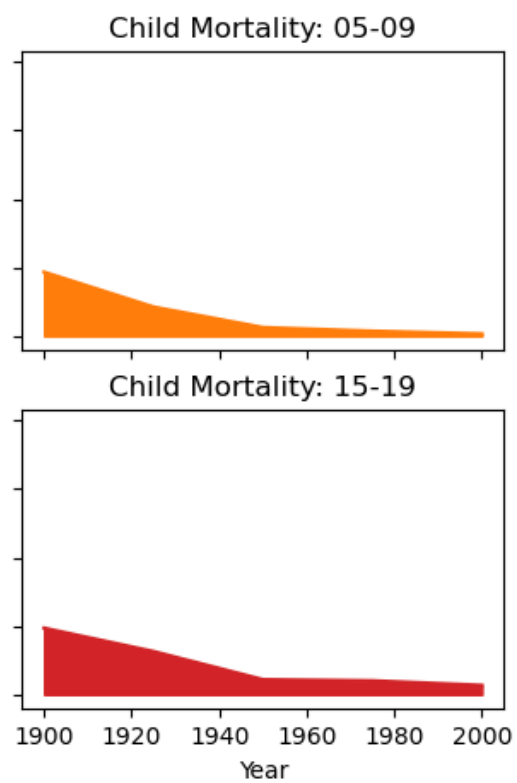
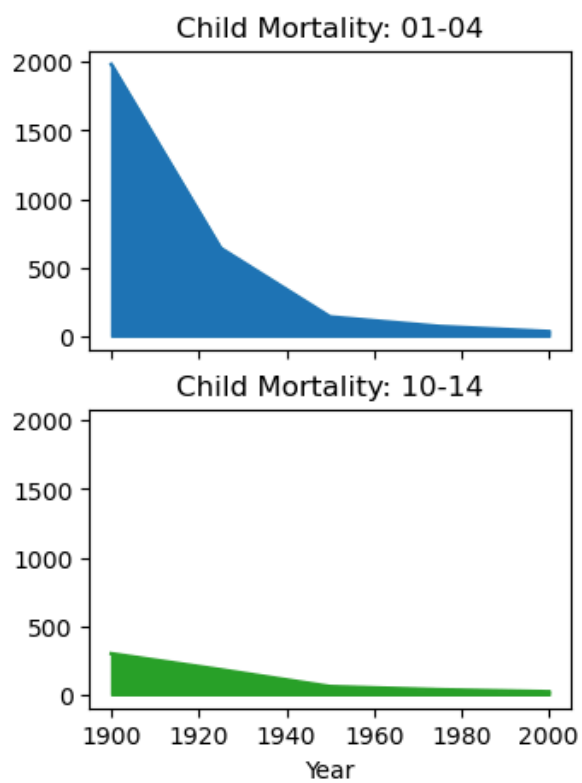
```
Out[32]: <Axes: ylabel='DeathRate'>
```



In [55]: *# Filter 'mortality_wide' for specific years (1900, 1925, 1950, 1975, 2000), the
for each age group ('01-04', '05-09', '10-14', '15-19') as separate subplots a
Each subplot is titled to reflect its respective child mortality age group, sh
for consistent comparison. The 'legend' is set to False to simplify visualizat
to optimize space and readability of the charts.*

```
mortality_wide.query('Year in (1900, 1925, 1950, 1975, 2000)').plot.area(
    title=['Child Mortality: 01-04', 'Child Mortality: 05-09',
          'Child Mortality: 10-14', 'Child Mortality: 15-19'],
    sharey=True, legend=False, subplots=True, layout=(2, 2), figsize=(8,5))
```

Out[55]: array([[<Axes: title={'center': 'Child Mortality: 01-04'}, xlabel='Year'>,
 <Axes: title={'center': 'Child Mortality: 05-09'}, xlabel='Year'>],
 [<Axes: title={'center': 'Child Mortality: 10-14'}, xlabel='Year'>,
 <Axes: title={'center': 'Child Mortality: 15-19'}, xlabel='Year'>]],
 dtype=object)



In []: